

Module 8 — Capstone

Lesson 8.1

Capstone project

Build something real that exercises subagents, MCP, customization, and automation. Self-grade against the rubric. Then ship it (or open-source it).

Why a capstone

The course has covered each technique in isolation. Real value comes from combining them. This project forces you to pick a problem in your work, design an end-to-end solution, and discover where the pieces actually fit.

You'll know the course landed when you're not asking "which feature do I use?" but "what's the right combination?"

The brief

Pick one problem from your work that is:

- **Recurring** — happens at least weekly
- **Currently manual** — costs time, attention, or both
- **Bounded** — has clear inputs and outputs
- **Tolerant of imperfection** — failed runs are recoverable, not catastrophic

Examples of capstone-shaped problems:

- Daily on-call brief that pulls open incidents from your tracker, recent deploys, and yesterday's error logs, then drafts a Slack post.
- PR review assistant that posts a structured review on every PR in a specific repo, tuned to your team's conventions.
- Doc drift detector that compares specific docs to current code weekly and opens issues for mismatches.
- Support-inbox triager that reads new tickets, suggests categories, drafts initial replies, and routes by topic.
- Release-notes generator that runs on every tag, produces user-facing notes, and posts a draft for human approval.

Pick one. Resist the temptation to scope it bigger.

Required components

To pass the rubric, your capstone must include:

1. A custom subagent

At least one specialized subagent (in `.claude/agents/`) tuned to part of your problem. Could be:

- A focused reviewer with your team's rubric
- A doc-comparator that produces structured output
- A triager with your team's category vocabulary

The point: you've codified domain knowledge into a reusable agent.

2. An MCP server (used or built)

Either:

- **Use an existing server** that connects you to a system your problem touches (Linear, GitHub, Slack, your DB)
- **Or build one** if no off-the-shelf option fits

If you use an existing server, document what tools you actually used and why.

3. Customization

Your project's `.claude/` should include at least:

- A tuned `CLAUDE.md` with the project's domain language
- A `.claude/settings.json` with appropriate permissions for the workflow
- One custom slash command that exercises the workflow interactively
- Optional: a hook that enforces some constraint specific to your flow

4. Automation

The project runs without human invocation in at least one mode. Pick:

- **Cron** — scheduled (daily digest, weekly summary)
- **Event-driven** — webhook (per-PR review, per-issue triage)
- **CI** — GitHub Action (per-commit doc check, per-PR review)

5. Observability

You can answer:

- How often does it run?
- What's the success rate?
- What does each run cost?
- When did it last fail, and why?

A JSONL log file + a 5-line summary script counts. A Grafana dashboard counts more.

6. A README

In your capstone repo, a README that covers:

- The problem and why it's worth automating
- The architecture (a small diagram)
- How to run it
- Cost estimate
- What you'd do differently next time

Rubric

Grade yourself honestly. Aim for at least 3/4 on every dimension.

Dimension	1 — basic	2 — works	3 — solid	4 — production-grade
Problem fit	"It does a thing"	Solves a real problem occasionally	Replaces a real recurring task	Indispensable; team would notice if it broke
Subagent quality	Generic; could be inline	Focused brief, tools constrained	Tuned to domain, well-described	Reusable across multiple capstones
MCP usage	Tools-as-resources confusion	Right primitive picked	Tight permissions, sanitized inputs	Auditable, scoped, secret-clean
Customization	CLAUDE.md only	Settings + commands	+ hook for hard constraint	+ composable for the team
Automation	Runs by hand	Cron or webhook works	Idempotent + retried	Feature-flagged + alerted
Observability	Print statements	Per-run log	+ cost tracking	+ dashboards + alerts
Documentation	A README exists	Steps to run	+ architecture + cost	+ handoff-ready

Time budget

Realistic estimate for a capstone done well: 1–2 weekends of focused time, spread across a week. If you're spending more, you've probably scope-crept; cut.

Common pitfalls

- **Scope too big** — "an agent that does our entire on-call workflow" is two months of work. Pick one slice.
- **Skipping observability** — without logs and metrics, you won't know if it's working. Build them in from day one.
- **Hand-tuning to one example** — make sure your prompts and agents handle realistic variety, not just the one issue you tested with.
- **Not running it in production** — the capstone isn't done until it's actually running on real data for a week.
- **Over-trusting** — agents fail unpredictably. Don't wire them into anything irreversible (auto-merge, auto-delete) without strong guards.

Sharing your capstone

If you've built something useful, consider:

- Sharing the agent and MCP server in your team's repos
- Open-sourcing the pieces that aren't proprietary
- Writing up what worked and what didn't (post-mortem post)

This is also good practice for explaining agentic systems to teammates who haven't taken the course.

What's next after the capstone

You've now exercised the full stack. From here:

- **Go wider** — apply the same patterns to other problems on your team
- **Go deeper** — read the [Anthropic docs](#) for newer features
- **Teach it** — the fastest way to consolidate the material is to walk someone else through it
- **Contribute** — to the MCP server ecosystem, to community subagents, to Claude Code itself

Welcome to using AI tools as serious infrastructure rather than novelties. Build well.

← Back to README | SYLLABUS